10 examples

1. CIS Controls v8 (Practical Cyber Hygiene)

1. **CIS 07.4:** Automatically trigger Snyk scans on every GitHub Pull Request.
2. **CIS 16.1:** Use SonarQube to enforce "Secure Coding" rules during the build.
3. **CIS 10.5:** Use Gitleaks to block commits containing AWS keys or database passwords.
4. **CIS 02.1:** Maintain an automated Software Bill of Materials (SBOM) via Trivy.
5. **CIS 04.1:** Verify that Docker images use "Distroless" or non-root users.
6. **CIS 05.4:** Force MFA for all developers accessing the Jenkins/GitLab pipeline.
7. **CIS 08.2:** Automatically alert when a "Critical" CVE is found in a production library.
8. **CIS 12.1:** Ensure all microservices communicate via encrypted TLS.
9. **CIS 13.3:** Use Data Loss Prevention (DLP) to scan for PII in log files.
10. **CIS 15.1:** Segment the "Dev" and "Prod" AWS environments completely.

2. ISO/IEC 27001:2022 (Governance & Policy)

1. **A.8.28:** Documented "Secure Coding Policy" enforced by SonarQube gates.
2. **A.8.8:** Automated "Technical Vulnerability Management" dashboard in your tool.
3. **A.8.29:** "Security Testing" evidence generated for every release.
4. **A.8.9:** Configuration management using Terraform (IaC) to prevent manual changes.
5. **A.8.15:** Centralized logging of all "Admin" actions in the CI/CD pipeline.
6. **A.5.15:** Role-Based Access Control (RBAC) for the GRC dashboard itself.
7. **A.8.31:** Strict separation between the Development and Production databases.
8. **A.8.12:** Automated scanning of code for sensitive data leakage.
9. **A.8.25:** Integrating security checks into the "System Development Life Cycle."
10. **A.5.36:** Generating a "Compliance Report" for auditors at the click of a button.

3. NIST CSF v2.0 (Risk & Identification)

1. **ID.AM-02:** Automated inventory of all software dependencies (via Snyk/Trivy).
2. **PR.PS-01:** Hardening the CI/CD server (Jenkins/GitHub Actions) configurations.
3. **ID.RA-01:** Mapping scan results to a "Risk Level" (High/Med/Low).
4. **DE.CM-01:** Continuous monitoring of the software supply chain.

5. **RS.MI-01:** Automated "Breaking the Build" if a critical vulnerability is found.
6. **GV.PO-01:** Aligning scan results with the organization's security policy.
7. **PR.DS-01:** Encrypting sensitive environment variables in the pipeline.
8. **DE.AE-01:** Detecting anomalous behavior in the build logs.
9. **RC.RP-01:** Automating the rollback of a deployment if a post-release bug is found.
10. **PR.AT-01:** Providing "Just-in-Time" security training to devs based on their code bugs.

4. ISO/IEC 27034 (Application Security focus)

1. **ONF:** Creating a centralized library of "Approved" secure Docker base images.
2. **ANF:** Defining specific security "Pass/Fail" criteria for each individual microservice.
3. **ASC 01:** Using SAST (SonarQube) to verify "Input Validation" requirements.
4. **ASC 05:** Verifying that the app uses secure communication protocols (HTTPS).
5. **ASC 07:** Automating "Assurance Predictions" (predicting if the app is safe to deploy).
6. **Trust Level:** Assigning a "Security Grade" (A, B, C) to applications based on scans.
7. **Security Activity:** Performing a "Threat Model" review before the first line of code.
8. **Data Format:** Ensuring sensitive data is handled in an encrypted memory format.
9. **Verification:** Using DAST tools to verify that the app is secure while running.
10. **Life Cycle:** Ensuring security controls move from "Dev" to "Ops" seamlessly.

5. NIST SP 800-218 (SSDF - Supply Chain)

1. **PW.1.1:** Integrating automated vulnerability scanners into the IDE.
2. **PW.4.1:** Enforcing "Peer Code Reviews" before any code is merged.
3. **RV.1.1:** Aggregating vulnerabilities from multiple tools into Parabellum IO.
4. **PS.1.1:** Digitally signing code artifacts to prevent tampering.
5. **PS.3.1:** Archiving old versions of code to allow for forensic audits.
6. **PO.1.1:** Defining "Security Champions" within the dev teams.
7. **PW.6.2:** Performing automated "Static Analysis" for every commit.
8. **RV.2.1:** Prioritizing fixes based on the "Exploitability" (EPSS) of a bug.
9. **PW.8.1:** Scanning Terraform scripts (IaC) for "Insecure Cloud Configs."
10. **PS.2.1:** Verifying the integrity of 3rd party libraries before downloading them.

ID	Technical Bug	Framework	Risk	Score	Fix
01	AWS Key in Code	CIS 5 / ISO A.8.28	95	0%	Rotate & Vault
02	SQL Injection	OWASP V5 / NIST SSDF	90	0%	Prepared Stmts
03	Old Library (CVE)	NIST ID.RA / ISO A.8.8	75	20%	Update Package
04	Public S3 Bucket	CIS 4 / NIST 800-53	88	0%	Private ACL

1. **Example: Hardcoded Secrets (Critical Failure)**

- **Vulnerability:** A Google Maps API key found in `javascript/map-init.js`.
- **Mapping: OWASP ASVS V3.1.1** (Secrets Management).
- **Compliance Score: 0%** (Automatic Fail).
- **Explanation:** "This violates the policy of 'No Secrets in Source Code.' Even if the key is restricted, it allows unauthorized usage costs."
- **Remediation:** Move the key to an Environment Variable (`process.env.MAPS_KEY`).

2. Example: Secure Input Handling (High Compliance / Success)

- **Vulnerability:** None. The code uses `PreparedStatement` for all database queries.
- **Mapping: NIST SSDF PW.6.2** (Safe Coding Practices).
- **Compliance Score: 100%**.
- **Explanation:** "By using parameterized queries, the developer has proactively neutralized SQL Injection risks, meeting the highest tier of NIST secure coding practices."

3. Example: Outdated Component (Moderate Risk)

- **Vulnerability:** Using `Axios 0.18.0` (has a Server-Side Request Forgery vulnerability).
- **Mapping: ISO 27001 A.8.8** (Tech Vulnerabilities).
- **Compliance Score: 40%** (Fail).
- **Explanation:** "Using libraries with known CVEs is a breach of technical vulnerability management policies."
- **Remediation:** Update to `Axios 1.6.0` via `npm update`.

Category 2: The "Container" Surface (Supply Chain)

4. Example: Root User Execution (Security Risk)

- **Vulnerability:** The Dockerfile does not specify a `USER`, so it runs as `root`.
- **Mapping: CIS Control 4.8** (Least Privilege).
- **Compliance Score: 20%**.
- **Explanation:** "If the container is compromised, the attacker has full root access to the container's OS. This violates the 'Principle of Least Privilege'."
- **Remediation:** Add `USER node` or `USER appuser` to the Dockerfile.

5. Example: Minimal Base Image (High Compliance / Success)

- **Vulnerability:** None. Developer used `node:alpine` instead of a full `ubuntu` image.
- **Mapping: CIS Control 7.1** (Reduce Attack Surface).
- **Compliance Score: 100%**.
- **Explanation:** "By using a 'distroless' or 'alpine' image, we reduce the number of installed packages (and thus the number of potential vulnerabilities) by 80%."

Category 3: The "Cloud/IaC" Surface (Infrastructure)

6. Example: Unencrypted S3 Bucket (Failure)

- **Vulnerability:** Terraform script defines an S3 bucket with `encryption = false`.
- **Mapping: NIST CSF PR.DS-01** (Data-at-Rest Protection).
- **Compliance Score: 0%**.
- **Explanation:** "Data at rest must be encrypted. This is a direct violation of data protection laws like GDPR or HIPAA."
- **Remediation:** Enable `server_side_encryption_configuration` in Terraform.

7. Example: Security Groups Open to 0.0.0.0 (Failure)

- **Vulnerability:** Port 22 (SSH) is open to the entire internet in the AWS Security Group.
- **Mapping: CIS Control 4.1** (Secure Config).
- **Compliance Score: 0%**.
- **Explanation:** "Exposing administrative ports to the public internet is an invitation for brute-force attacks."
- **Remediation:** Restrict SSH access to a specific "Bastion Host" or Company IP range.

8. Example: Automated Backup Enabled (High Compliance / Success)

- **Vulnerability:** None. Terraform includes `backup_retention_period = 7`.
- **Mapping: ISO 27001 A.8.13** (Information Backup).
- **Compliance Score: 100%**.
- **Explanation:** "This proves the infrastructure is resilient. Even if a ransomware attack occurs, we have a compliance-ready recovery path."

Category 4: The "Pipeline" Surface (Orchestration)

9. Example: Unsigned Commits (Process Failure)

- **Vulnerability:** Code is being pushed to `main` without GPG signatures.
- **Mapping: NIST SSDF PS.1.1** (Verify Software Integrity).
- **Compliance Score: 0%**.
- **Explanation:** "Without signed commits, we cannot verify the identity of the developer, leading to a risk of 'Insider Threat' or 'Impersonation'."
- **Remediation:** Enforce "Require Signed Commits" in GitHub/GitLab settings.

10. Example: Automated Gatekeeping (High Compliance / Success)

- **Vulnerability:** None. The CI/CD pipeline is configured to "Fail" if SonarQube finds a "Blocker" issue.
- **Mapping: ISO 27034-7** (Security Assurance).
- **Compliance Score: 100%**.
- **Explanation:** "This is the 'Automated GRC' in action. The tool ensures that non-compliant code can *never* reach production, satisfying the 'Continuous Assurance' requirement of ISO 27034."

1. Example: Hardcoded Secrets (Critical Failure)

- **Vulnerability:** A Google Maps API key found in `javascript/map-init.js`.
- **Mapping: OWASP ASVS V3.1.1** (Secrets Management).
- **Compliance Score: 0%** (Automatic Fail).

- **Explanation:** "This violates the policy of 'No Secrets in Source Code.' Even if the key is restricted, it allows unauthorized usage costs."
- **Remediation:** Move the key to an Environment Variable (`process.env.MAPS_KEY`).

2. Example: Secure Input Handling (High Compliance / Success)

- **Vulnerability:** None. The code uses `PreparedStatement` for all database queries.
- **Mapping: NIST SSDF PW.6.2** (Safe Coding Practices).
- **Compliance Score: 100%**.
- **Explanation:** "By using parameterized queries, the developer has proactively neutralized SQL Injection risks, meeting the highest tier of NIST secure coding practices."

3. Example: Outdated Component (Moderate Risk)

- **Vulnerability:** Using `Axios 0.18.0` (has a Server-Side Request Forgery vulnerability).
- **Mapping: ISO 27001 A.8.8** (Tech Vulnerabilities).
- **Compliance Score: 40%** (Fail).
- **Explanation:** "Using libraries with known CVEs is a breach of technical vulnerability management policies."
- **Remediation:** Update to `Axios 1.6.0` via `npm update`.

Category 2: The "Container" Surface (Supply Chain)

4. Example: Root User Execution (Security Risk)

- **Vulnerability:** The Dockerfile does not specify a `USER`, so it runs as `root`.
- **Mapping: CIS Control 4.8** (Least Privilege).
- **Compliance Score: 20%**.
- **Explanation:** "If the container is compromised, the attacker has full root access to the container's OS. This violates the 'Principle of Least Privilege'."
- **Remediation:** Add `USER node` or `USER appuser` to the Dockerfile.

5. Example: Minimal Base Image (High Compliance / Success)

- **Vulnerability:** None. Developer used `node:alpine` instead of a full `ubuntu` image.
- **Mapping: CIS Control 7.1** (Reduce Attack Surface).
- **Compliance Score: 100%**.

- **Explanation:** "By using a 'distroless' or 'alpine' image, we reduce the number of installed packages (and thus the number of potential vulnerabilities) by 80%."

Category 3: The "Cloud/IaC" Surface (Infrastructure)

6. Example: Unencrypted S3 Bucket (Failure)

- **Vulnerability:** Terraform script defines an S3 bucket with `encryption = false`.
- **Mapping: NIST CSF PR.DS-01** (Data-at-Rest Protection).
- **Compliance Score: 0%**.
- **Explanation:** "Data at rest must be encrypted. This is a direct violation of data protection laws like GDPR or HIPAA."
- **Remediation:** Enable `server_side_encryption_configuration` in Terraform.

7. Example: Security Groups Open to 0.0.0.0 (Failure)

- **Vulnerability:** Port 22 (SSH) is open to the entire internet in the AWS Security Group.
- **Mapping: CIS Control 4.1** (Secure Config).
- **Compliance Score: 0%**.
- **Explanation:** "Exposing administrative ports to the public internet is an invitation for brute-force attacks."
- **Remediation:** Restrict SSH access to a specific "Bastion Host" or Company IP range.

8. Example: Automated Backup Enabled (High Compliance / Success)

- **Vulnerability:** None. Terraform includes `backup_retention_period = 7`.
- **Mapping: ISO 27001 A.8.13** (Information Backup).
- **Compliance Score: 100%**.
- **Explanation:** "This proves the infrastructure is resilient. Even if a ransomware attack occurs, we have a compliance-ready recovery path."

Category 4: The "Pipeline" Surface (Orchestration)

9. Example: Unsigned Commits (Process Failure)

- **Vulnerability:** Code is being pushed to `main` without GPG signatures.
- **Mapping: NIST SSDF PS.1.1** (Verify Software Integrity).
- **Compliance Score: 0%**.

- **Explanation:** "Without signed commits, we cannot verify the identity of the developer, leading to a risk of 'Insider Threat' or 'Impersonation'."
- **Remediation:** Enforce "Require Signed Commits" in GitHub/GitLab settings.

10. Example: Automated Gatekeeping (High Compliance / Success)

- **Vulnerability:** None. The CI/CD pipeline is configured to "Fail" if SonarQube finds a "Blocker" issue.
- **Mapping: ISO 27034-7** (Security Assurance).
- **Compliance Score: 100%**.
- **Explanation:** "This is the 'Automated GRC' in action. The tool ensures that non-compliant code can *never* reach production, satisfying the 'Continuous Assurance' requirement of ISO 27034."

DevSecOps Surface	**Technical Finding (Vulnerability)**	**Framework Mapping**	**Policy Violated**	**Risk Score**	**Compliance Score**	**Remediation Action**
CODE (SAST)	**Hardcoded API Key**in auth.js	**OWASP ASVS V3.1**	Secrets Mgmt Policy	**95/100**	**0% (FAIL)**	Move to Env Vars / Vault
CODE (SAST)	**Parameterized Queries** used	**NIST SSDF PW.6.2**	Input Validation	**5/100**	**100% (PASS)**	*None Required (Secure)*
CODE (SCA)	**Outdated Library**(CVE-2023)	**ISO 27001 A.8.8**	Vulnerability Mgmt	**72/100**	**20% (FAIL)**	npm update to patched version
CONTAINER	**Running as Root** in Dockerfile	**CIS Control 4.8**	Least Privilege	**65/100**	**10% (FAIL)**	Add USER nonroottoto Dockerfile

CONTAINER	**Distroless/Alpine Image** used	**CIS Control 7.1**	Attack Surface Reduction	**10/100**	**100% (PASS)**	*None Required (Optimized)*
INFRA (IaC)	**Public S3 Bucket**via Terraform	**NIST CSF PR.DS-01**	Data Protection Policy	**88/100**	**0% (FAIL)**	Set acl = "private"
INFRA (IaC)	**SSH Open to 0.0.0.0/0**	**CIS Control 4.1**	Network Hardening	**90/100**	**0% (FAIL)**	Restrict to Specific IP Range
INFRA (IaC)	**Automated Backups** Enabled	**ISO 27001 A.8.13**	Business Continuity	**5/100**	**100% (PASS)**	*None Required (Resilient)*
PIPELINE	**Unsigned Git Commits**	**NIST SSDF PS.1.1**	Integrity Policy	**40/100**	**0% (FAIL)**	Enforce GPG Signing in Git
PIPELINE	**Automated Sec-Gates** Active	**ISO 27034-7**	Continuous Assurance	**5/100**	**100% (PASS)**	*None Required (Automated)*

Framework / Standard	**Total Scope**	**Project Focus**	**Key Mapping Example**
CIS Controls v8	153 Safeguards	**~25 Automated**	**CIS 07.4:** Automated Vuln. Scanning

ISO/IEC 27001:2022	93 Controls	**~15 Tech Controls**	**A.8.28:** Secure Coding Policy
NIST CSF v2.0	106 Sub-cats	**~20 Mappings**	**ID.RA-01:** Risk Identification
ISO/IEC 27034	7 Parts	**ANF Component**	**27034-1:** App Security Validation
NIST SSDF (800-218)	19 Practices	**Auto Evidence**	**RV.1.1:** Verification of Code Safety

DevSecOps Surface	Technical Finding	Primary Framework	Policy Violated	Risk Score	Compliance Score	Remediation Action
CODE (SAST)	**Hardcoded API Key**	**ISO 27001 A.8.28**	Secure Coding Policy	**95/100**	**0% (FAIL)**	Rotate & move to Vault
CODE (SAST)	**Secure Input Handling**	**NIST SSDF PW.6.2**	Input Validation	**5/100**	**100% (PASS)**	*Proactive Security*
CODE (SCA)	**Outdated Library**	**NIST CSF ID.RA-01**	Vulnerability Mgmt	**72/100**	**20% (FAIL)**	npm updatelibrary
CONTAINER	**Running as Root**	**CIS Control 4.8**	Least Privilege	**65/100**	**10% (FAIL)**	Add USER to Dockerfile

CONTAINER	**Alpine Image used**	**CIS Control 7.1**	Attack Surface Reduc.	**10/100**	**100% (PASS)**	*Optimized Base Image*
INFRA (IaC)	**Public S3 Bucket**	**ISO 27034 (ASC)**	Data Protection	**88/100**	**0% (FAIL)**	Set acl = "private"
INFRA (IaC)	**SSH Open to 0.0.0.0**	**CIS Control 4.1**	Network Hardening	**90/100**	**0% (FAIL)**	Limit IP Whitelist
INFRA (IaC)	**Daily Backups**	**ISO 27001 A.8.13**	Business Continuity	**5/100**	**100% (PASS)**	*Resilience Verified*
PIPELINE	**Unsigned Commits**	**NIST SSDF PS.1.1**	Software Integrity	**40/100**	**0% (FAIL)**	Enforce GPG Signing
PIPELINE	**Sec-Gate Active**	**ISO 27034-7**	Continuous Assurance	**5/100**	**100% (PASS)**	*Automated Control*